

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

SIOPTR: TT10: Week 5

My Own KERnel (MOKER) version: Framework Mutex

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 Resource Shared Mechanism

It will be added a new resource shared mechanism to the MOKER framework, `MUTEX_LIFO`. This new mechanism is synchronization mechanism, similar to a mutex (**M**utual **e**xclusion object). This is a synchronization mechanism for enforcing that only one process (or thread) accesses to system resource (shared).

This new mechanism is automatically created by the system at the boot. There is only one `MUTEX_LIFO` on the system. It is not possible to create new ones.

When a process needs to access a system shared resource must lock the mutex from the other processes while using such resource. The mutex is set to unlock when the resource is no longer needed.

Whenever more than one process contend for the mutex, one process locks it and the others are blocked. When the mutex is unlocked the blocked processes are unblocked according to the LIFO policy. That is, the last process to be blocked it is the first to be unblocked. Further, only one process is unblocked at single time.

2 Configuration Option Entry

It must be created the `MOKER_MUTEX_LIFO` configuration entry that depends on `MOKER_FRAMEWORK` entry.

Open the `linux-6.19.9-moker/kernel/moker/Kconfig` file and update it by adding (blue text):

```
menu "MOKER framework"
config MOKER_FRAMEWORK
bool "My Own Kernel Framework"
default y
...

config MOKER_MUTEX_LIFO
bool "MOKER mutex policy: LIFO"
default y
depends on MOKER_FRAMEWORK

endmenu
```

3 Mutex coding

Create a new file called `lf_mutex.h` in the `linux-6.19.9-moker/kernel/moker` directory. The content of `lf_mutex.h` is:

```

#ifndef __LF_MUTEX_H_
#define __LF_MUTEX_H_

#include <linux/sched.h>
#include <linux/spinlock.h>
#include <linux/list.h>
#include <asm/atomic.h>

struct lf_mutex_wq{
    struct list_head tasks;
    atomic_t flag;
    raw_spinlock_t lock;
};

struct lf_mutex_node{
    struct list_head node;
    struct task_struct *task;
};

void init_lf_mutex(void);
void lock_lf_mutex(void);
void unlock_lf_mutex(void);
int enqueue_lf_mutex_task(struct task_struct *p);
struct task_struct * dequeue_lf_mutex_task(void);
#endif

```

It is created two data structures, `struct lf_mutex_wq` and `struct lf_mutex_node`.

The `struct lf_mutex_wq` is the mutex wait queue (queue for blocked process) and the `struct lf_mutex_node` is used for the blocked processes into the queue.

The `struct list_head tasks` field is the head of the blocked processes queue. The `raw_spinlock_t lock` field is used to control the queue operations. The `atomic_t flag` field can assume only two values, zero or one. If the value is one it means that the mutex is locked otherwise it is unlocked.

The `struct list_head node` it is a node on the queue and `struct task_struct *task` it is a pointer to the the blocked process.

Create a new file called `lf_mutex.c` in the `linux-6.19.9-moker/kerne`l/`moker` directory. The content of `lf_mutex.c` is:

```

#include <linux/sched.h>
#include <linux/slab.h>
#include <asm/uaccess.h>

#include "lf_mutex.h"

struct lf_mutex_wq wq;

void init_lf_mutex(void){
    INIT_LIST_HEAD(&wq.tasks);
    raw_spin_lock_init(&wq.lock);
    atomic_set(&wq.flag, 0);
}

void lock_lf_mutex(){
    struct task_struct *p = current;
    while(!atomic_add_unless(&wq.flag, 1, 1)){
        set_current_state(TASK_INTERRUPTIBLE);
        enqueue_lf_mutex_task(p);
        schedule();
    }
}

void unlock_lf_mutex(void){
    struct task_struct *p = current;
    struct task_struct *t = NULL;
    t = dequeue_lf_mutex_task();
    if(t){
        if(!wake_up_process(t)){
            printk(KERN_ERR "BUG: wake up process failed: %d\n", t->pid);
        }
    }
    atomic_set(&wq.flag, 0);
}

int enqueue_lf_mutex_task(struct task_struct *p)
{
    int ret = -1;
    struct lf_mutex_node *t = kmalloc(sizeof(struct lf_mutex_node), GFP_KERNEL);
    if(t){
        t->task = p;
        raw_spin_lock(&wq.lock);
        list_add(&t->node, &wq.tasks);
        raw_spin_unlock(&wq.lock);
        ret = 0;
    }
    return ret;
}

struct task_struct * dequeue_lf_mutex_task(void)
{
    struct task_struct *p = NULL;
    struct lf_mutex_node *t = NULL;
    raw_spin_lock(&wq.lock);
    if(!list_empty(&wq.tasks)){
        t = list_first_entry(&wq.tasks, struct lf_mutex_node, node);
        p = t->task;
        list_del(&t->node);
    }
    raw_spin_unlock(&wq.lock);
    if(t){
        kfree(t);
    }
    return p;
}

```

`struct lf_mutex_wq wq` is a global variable for the mutex. This variable is created at the system boot.

4 Initialize MUTEX

To create the struct `lf_mutex_wq wq`, this must be known by the system.

Put this data structure visible to all kernel, by updating the `linux-6.19.9-moker/kernel/sched/sched.h`

```
...
#ifdef CONFIG_PARAVIRT
# include <asm/paravirt.h>
# include <asm/paravirt_api_clock.h>
#endif

#include <asm/barrier.h>

#include "cpupri.h"
#include "cpudeadline.h"

#ifdef CONFIG_MOKER_TRACING
#include "../moker/trace.h"
#endif
#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
# include "../moker/lf_rq.h"
#endif

#ifdef CONFIG_MOKER_MUTEX_LIFO
# include "../moker/lf_mutex.h"
#endif

/* task_struct::on_rq states: */
#define TASK_ON_RQ_QUEUED 1
#define TASK_ON_RQ_MIGRATING 2
...
```

Then initialize the mutex by invoking the `init_lf_mutex()`. For that update the `sched_init` function, in `linux-6.19.9-moker/kernel/sched/core.c` file.

```

void __init sched_init(void)
{
    unsigned long ptr = 0;
    int i;

    ...

#ifdef CONFIG_RT_GROUP_SCHED
    init_rt_bandwidth(&root_task_group.rt_bandwidth,
        global_rt_period(), global_rt_runtime());
#endif /* CONFIG_RT_GROUP_SCHED */

#ifdef CONFIG_CGROUP_SCHED
    task_group_cache = KMEM_CACHE(task_group, 0);

    list_add(&root_task_group.list, &task_groups);
    INIT_LIST_HEAD(&root_task_group.children);
    INIT_LIST_HEAD(&root_task_group.siblings);
    autogroup_init(&init_task);
#endif /* CONFIG_CGROUP_SCHED */

#ifdef CONFIG_MOKER_MUTEX_LIFO
    init_lf_mutex();
#endif

    for_each_possible_cpu(i) {
        struct rq *rq;

        rq = cpu_rq(i);
        raw_spin_lock_init(&rq->__lock);
        ...
    }
    ...
}

```

5 Create new system calls

It will be needed two new system calls. One to lock and other to unlock. Update the linux-6.19.9-moker/arch/x86/entry/syscalls/syscall_64.tbl file.

```

...
443 common quotactl_fd sys_quotactl_fd
444 common landlock_create_ruleset sys_landlock_create_ruleset
445 common landlock_add_rule sys_landlock_add_rule
446 common landlock_restrict_self sys_landlock_restrict_self
447 common memfd_secret sys_memfd_secret
448 common process_mrelease sys_process_mrelease
449 common futex_waitv sys_futex_waitv
450 common set_mempolicy_home_node sys_set_mempolicy_home_node

#ifdef CONFIG_MOKER_FRAMEWORK
451 common mocker_tracing sys_mocker_tracing
452 common mocker_mutex_lock sys_mocker_mutex_lock
453 common mocker_mutex_unlock sys_mocker_mutex_unlock
#endif
#
# Due to a historical design error, certain syscalls are numbered differently
# in x32 as compared to native x86_64. These syscalls have numbers 512-547.
# Do not add new syscalls to this range. Numbers 548 and above are available
# for non-x32 use.
...

```

Update `syscalls.c` and `syscalls.h` files into the `linux-6.19.9-moker/kernel/moker` directory.

```

#ifndef __SYSCALLS_H
#define __SYSCALLS_H
int sys_mocker_tracing (unsigned int enable);
int sys_mocker_mutex_lock(void);
int sys_mocker_mutex_unlock(void);
#endif

```

```

#include <linux/syscalls.h>
#include "../sched/sched.h"
#include "trace.h"
#include "lf_mutex.h"
SYSCALL_DEFINE1(moker_tracing, unsigned int, enable)
{
    printk("MOKER: moker_tracing:[%d][%d]\n", (int) enable, current->pid);
    return sys_moker_tracing(enable);
}
int sys_moker_tracing (unsigned int enable){
    #ifdef CONFIG_MOKER_TRACING
        printk("MOKER: sys_moker_tracing:[%d][%d]\n", (int) enable, current->pid);
        trace_enable(enable);
    #endif
    return 0;
}
SYSCALL_DEFINE0(moker_mutex_lock)
{
    return sys_moker_mutex_lock();
}
int sys_moker_mutex_lock (){
    #ifdef CONFIG_MOKER_MUTEX_LIFO
        lock_lf_mutex();
    #endif
    return 0;
}
SYSCALL_DEFINE0(moker_mutex_unlock)
{
    return sys_moker_mutex_unlock();
}
int sys_moker_mutex_unlock (){
    #ifdef CONFIG_MOKER_MUTEX_LIFO
        unlock_lf_mutex();
    #endif
    return 0;
}

```

6 Update MOKER Tracing Mechanism

Update `linux-6.19.9-moker/kernel/moker/trace.h` for collecting four new events. The enqueue to the wait queue (or queue that hold blocked processes) event and the dequeue from such queue event. The other two events are related to the mutex lock and unlock.

```

#ifndef __TRACE_H_
#define __TRACE_H_

#define TRACE_ENTRY_NAME "moker_trace"
#define TRACE_BUFFER_SIZE 1000
#define TRACE_STRING_BUFFER_SIZE 200
#define TRACE_TASK_COMM_LEN 16

enum evt{
    SCHED_TICK = 0,
    SWITCH_AWAY,
    SWITCH_TO,
    ENQUEUE_RQ,
    DEQUEUE_RQ,

    ENQUEUE_WQ,
    DEQUEUE_WQ,
    MUTEX_LOCK,
    MUTEX_UNLOCK,
};

...
#endif

```

Update `linux-6.19.9-moker/kernel/moker/trace.c` to encompass the mutex related events.

```

...
static int dequeue (char *buffer)
{
    int ret = 0, len;
    char evt[10];
    spin_lock(&trace.lock);
    if (!is_empty(trace.read_item, trace.write_item)) { //if it is not empty

        switch((int)trace.events[trace.read_item].event){
            case SCHED_TICK:
                strcpy(evt, "SCH_TK");
                break;
            case SWITCH_AWAY:
                strcpy(evt, "SWT_AY");
                break;
            case SWITCH_TO:
                strcpy(evt, "SWT_TO");
                break;
            case ENQUEUE_RQ:
                strcpy(evt, "ENQ_RQ");
                break;
            case DEQUEUE_RQ:
                strcpy(evt, "DEQ_RQ");
                break;
            case ENQUEUE_WQ:
                strcpy(evt, "ENQ_WQ");
                break;
            case DEQUEUE_WQ:
                strcpy(evt, "DEQ_WQ");
                break;
            case MUTEX_LOCK:
                strcpy(evt, "MUT_LK");
                break;
            case MUTEX_UNLOCK:
                strcpy(evt, "MUT_UL");
                break;

            default:
                strcpy(evt, "UK_EVT");
        }
        len = sprintf(buffer, "%llu, ", trace.events[trace.read_item].time);
        len += sprintf(buffer + len, "%s, ", evt);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].policy);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].prio);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].pid);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].state);
        len += sprintf(buffer + len, "%s\n", trace.events[trace.read_item].comm);

        increment(&trace.read_item);
        ret = 1;
    }
    spin_unlock(&trace.lock);
    return ret;
}
...

```

Update linux-6.19.9-moker/kernel/moker/lf_mutex.c file.

```

#include <linux/sched.h>
#include <linux/slab.h>
#include <asm/uaccess.h>

#include "lf_mutex.h"

#include "trace.h"

...
void lock_lf_mutex(){
    struct task_struct *p = current;
    while(!atomic_add_unless(&wq.flag, 1, 1)){
        set_current_state(TASK_INTERRUPTIBLE);
        enqueue_lf_mutex_task(p);
        schedule();
    }
}

#ifdef CONFIG_MOKER_TRACING
    moker_trace(MUTEX_LOCK, -1, p);
#endif
}

void unlock_lf_mutex(void){
    struct task_struct *p = current;
    struct task_struct *t = NULL;
    t = dequeue_lf_mutex_task();
    if(t){
        if(!wake_up_process(t)){
            printk(KERN_ERR "BUG: wake up process failed: %d\n", t->pid);
        }
    }
    atomic_set(&wq.flag, 0);

#ifdef CONFIG_MOKER_TRACING
    moker_trace(MUTEX_UNLOCK, -1, p);
#endif
}

int enqueue_lf_mutex_task(struct task_struct *p)
{
    int ret = -1;
    struct lf_mutex_node *t = kmalloc(sizeof(struct lf_mutex_node), GFP_KERNEL);
    if(t){
        t->task = p;
        raw_spin_lock(&wq.lock);
        list_add(&t->node, &wq.tasks);
        raw_spin_unlock(&wq.lock);
        ret = 0;

#ifdef CONFIG_MOKER_TRACING
        moker_trace(ENQUEUE_WQ, -1, p);
#endif
    }
    return ret;
}

struct task_struct * dequeue_lf_mutex_task(void)
{
    struct task_struct *p = NULL;
    struct lf_mutex_node *t = NULL;
    raw_spin_lock(&wq.lock);
    if(!list_empty(&wq.tasks)){
        t = list_first_entry(&wq.tasks, struct lf_mutex_node, node);
        p = t->task;
        list_del(&t->node);
    }
    raw_spin_unlock(&wq.lock);
    if(t){
        kfree(t);
    }

#ifdef CONFIG_MOKER_TRACING
    if(p)
        moker_trace(DEQUEUE_WQ, -1, p);
#endif
    return p;
}

```

7 Compiling

For compiling all this code update the `linux-6.19.9-moker/kernel/moker/Makefile` file

```
# MOKER framework makefile

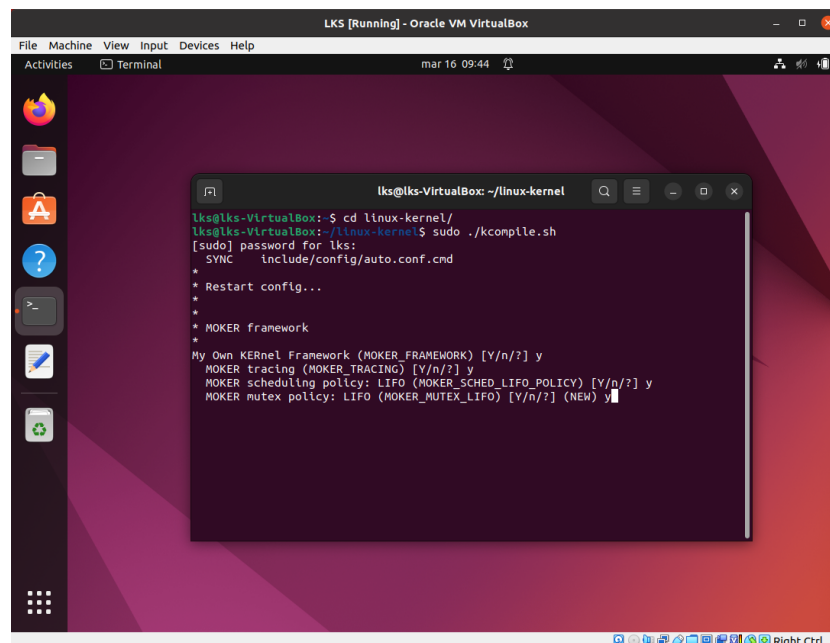
obj-$(CONFIG_MOKER_TRACING) += trace.o
obj-y += syscalls.o
obj-$(CONFIG_MOKER_SCHED_LIFO_POLICY) += sched.o lf_rq.o

obj-$(CONFIG_MOKER_MUTEX_LIFO) += lf_mutex.o
```

Compile the Linux kernel:

```
sudo ./kcompile.sh
```

We have added a new configuration option to the `moker/Kconfig` file and we do not execute the `make menuconfig` tool to create a new `.config` file. The `kbuild` system detects such change and asks what is our choice for that option. In this case type `y` and then `enter` key.



```
sudo reboot
```

8 Testing

Open a terminal and placed into the `task` directory that contains the testing programs: `launcher` and `task`.

Update the `defs.h` file

```
#ifndef DEFS_H_
#define DEFS_H_

...

#define SCHED_LIFO 8

#define SYS_MOKER_TRACING_ENABLE 471
#define SYS_MOKER_ID 472

#define SYS_MOKER_MUTEX_LOCK 473
#define SYS_MOKER_MUTEX_UNLOCK 474

struct task {
    int id;
    unsigned long long C; //exec
    unsigned long long T; //period
    unsigned long long O; //first job offset
    unsigned int njobs; //number of jobs
};

#endif
```

Update the `task.c` file

```

int main(int argc, char** argv)
{
    ...

    release = time0 + 0;

    for(i=0;i<njobs;i++){
        r.tv_sec = release / NSEC_PER_SEC;
        r.tv_nsec = release % NSEC_PER_SEC;

        printf("Task(%d,%d,%d): sleeping until %lld\n",task_id,getpid(),i,release);
        clock_nanosleep(CLOCK_MONOTONIC,TIMER_ABSTIME, &r,NULL);
        printf("Task(%d,%d,%d): ready for execution\n",task_id,getpid(),i);

        syscall(SYS_MOKER_MUTEX_LOCK);
        printf("Task(%d,%d,%d): entering critical section\n",task_id,getpid(),i);

        do_work(C);

        printf("Task(%d,%d,%d): leaving critical section\n",task_id,getpid(),i);
        syscall(SYS_MOKER_MUTEX_UNLOCK);

        //computes the next release
        release += T;
    }

    exit(task_id);
    return 0;
}

```

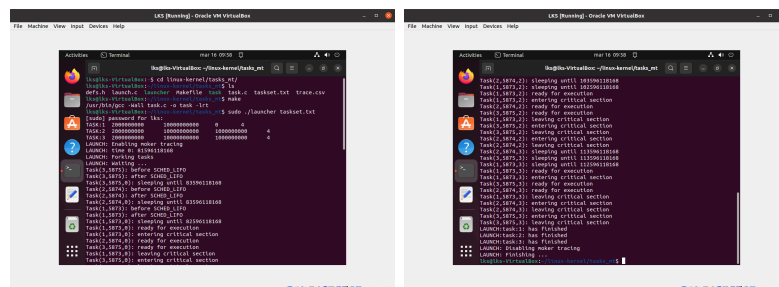
1. Compile it by invoking:
make
2. Change the taskset.txt file to.

```

1,2000000000,10000000000,0,4,
2,2000000000,10000000000,1000000000,4,
3,2000000000,10000000000,1000000000,4,

```

3. Put the set of tasks running:
sudo ./launcher taskset.txt



4. Wait and get the scheduling trace:
cat /proc/moker_trace > trace.csv

5. Check output
gedit trace.csv

```
1 76596584869,ENQ_RQ,7,120,5875,0,task
2 76596590808,DEQ_RQ,7,120,5875,1,task
3 76596591391,SWT_AY,7,120,5875,1,task
4 76596591426,SWT_TO,0,120,23,0,kworker/u2:1
5 76596883791,ENQ_RQ,7,120,5874,0,task
6 76596890146,DEQ_RQ,7,120,5874,1,task
7 76596890380,SWT_AY,7,120,5874,1,task
8 76596890417,SWT_TO,0,120,23,0,kworker/u2:1
9 76597160877,ENQ_RQ,7,120,5873,0,task
10 76597166907,DEQ_RQ,7,120,5873,1,task
11 76597167108,SWT_AY,7,120,5873,1,task
12 76597167145,SWT_TO,0,120,23,0,kworker/u2:1
13 82596222832,ENQ_RQ,7,120,5873,1,task
14 82596232389,SWT_AY,0,120,0,0,swapper
15 82596232531,SWT_TO,7,120,5873,0,task
16 82596257656,MUT_LK,7,120,5873,0,task
17 83603490173,ENQ_RQ,7,120,5875,1,task
18 83603492506,ENQ_RQ,7,120,5874,1,task
19 83603519603,SWT_AY,7,120,5873,0,task
20 83603519798,SWT_TO,7,120,5874,0,task
21 83603566186,ENQ_WQ,7,120,5874,1,task
22 83603566418,DEQ_RQ,7,120,5874,1,task
23 83603566550,SWT_AY,7,120,5874,1,task
24 83603566584,SWT_TO,7,120,5875,0,task
25 83603576995,ENQ_WQ,7,120,5875,1,task
26 83603577103,DEQ_RQ,7,120,5875,1,task
27 83603577209,SWT_AY,7,120,5875,1,task
28 83603577243,SWT_TO,7,120,5873,0,task
29 83647129454,DEQ_WQ,7,120,5875,1,task
30 83647129559,ENQ_RQ,7,120,5875,1,task
31 83647129778,MUT_UL,7,120,5873,0,task
32 83647129919,SWT_AY,7,120,5873,0,task
33 83647129947,SWT_TO,7,120,5875,0,task
34 83647130751,MUT_LK,7,120,5875,0,task
35 84704190011,DEQ_WQ,7,120,5874,1,task
36 84704190525,ENQ_RQ,7,120,5874,1,task
37 84704191053,MUT_UL,7,120,5875,0,task
38 84704192560,SWT_AY,7,120,5875,0,task
39 84704192809,SWT_TO,7,120,5874,0,task
40 84704196221,MUT_LK,7,120,5874,0,task
41 85751226405,MUT_UL,7,120,5874,0,task
42 85751235521,DEQ_RQ,7,120,5874,1,task
43 85751236543,SWT_AY,7,120,5874,1,task
44 85751236771,SWT_TO,7,120,5875,0,task
45 85751251881,DEQ_RQ,7,120,5875,1,task
46 85751252005,SWT_AY,7,120,5875,1,task
47 85751252056,SWT_TO,7,120,5873,0,task
48 85751264446,DEQ_RQ,7,120,5873,1,task
49 85751267595,SWT_AY,7,120,5873,1,task
50 85751267638,SWT_TO,0,120,348,0,NetworkManager
51 92596217331,ENQ_RQ,7,120,5873,1,task
52 92596230247,SWT_AY,0,120,0,0,swapper
53 92596230365,SWT_TO,7,120,5873,0,task
54 92596263117,MUT_LK,7,120,5873,0,task
```

55	93597128977,ENQ_RQ,7,120,5874,1,task
56	93597130524,ENQ_RQ,7,120,5875,1,task
57	93597157811,SWT_AY,7,120,5873,0,task
58	93597158008,SWT_TO,7,120,5875,0,task
59	93597190324,ENQ_WQ,7,120,5875,1,task
60	93597190528,DEQ_RQ,7,120,5875,1,task
61	93597190664,SWT_AY,7,120,5875,1,task
62	93597190699,SWT_TO,7,120,5874,0,task
63	93597202021,ENQ_WQ,7,120,5874,1,task
64	93597202185,DEQ_RQ,7,120,5874,1,task
65	93597202291,SWT_AY,7,120,5874,1,task
66	93597202327,SWT_TO,7,120,5873,0,task
67	93647225722,DEQ_WQ,7,120,5874,1,task
68	93647225834,ENQ_RQ,7,120,5874,1,task
69	93647225920,MUT_UL,7,120,5873,0,task
70	93647226097,SWT_AY,7,120,5873,0,task
71	93647226129,SWT_TO,7,120,5874,0,task
72	93647226717,MUT_LK,7,120,5874,0,task
73	94691859037,DEQ_WQ,7,120,5875,1,task
74	94691859644,ENQ_RQ,7,120,5875,1,task
75	94691859961,MUT_UL,7,120,5874,0,task
76	94691861582,SWT_AY,7,120,5874,0,task
77	94691861775,SWT_TO,7,120,5875,0,task
78	94691864267,MUT_LK,7,120,5875,0,task
79	95743626235,MUT_UL,7,120,5875,0,task
80	95743633674,DEQ_RQ,7,120,5875,1,task
81	95743634624,SWT_AY,7,120,5875,1,task
82	95743634835,SWT_TO,7,120,5874,0,task
83	95743648341,DEQ_RQ,7,120,5874,1,task
84	95743648465,SWT_AY,7,120,5874,1,task
85	95743648514,SWT_TO,7,120,5873,0,task
86	95743661593,DEQ_RQ,7,120,5873,1,task
87	95743664165,SWT_AY,7,120,5873,1,task
88	95743664199,SWT_TO,0,120,308,0,kworker/u2:5
89	102609283866,ENQ_RQ,7,120,5873,1,task
90	102609316219,SWT_AY,0,139,5990,0,lsb_release
91	102609316289,SWT_TO,7,120,5873,0,task
92	102609329077,MUT_LK,7,120,5873,0,task
93	103608520066,ENQ_RQ,7,120,5875,1,task
94	103608521030,ENQ_RQ,7,120,5874,1,task
95	103608548248,SWT_AY,7,120,5873,0,task
96	103608548449,SWT_TO,7,120,5874,0,task
97	103608577026,ENQ_WQ,7,120,5874,1,task
98	103608577614,DEQ_RQ,7,120,5874,1,task
99	103608577746,SWT_AY,7,120,5874,1,task
100	103608577779,SWT_TO,7,120,5875,0,task
101	103608588623,ENQ_WQ,7,120,5875,1,task
102	103608588813,DEQ_RQ,7,120,5875,1,task
103	103608588919,SWT_AY,7,120,5875,1,task
104	103608588953,SWT_TO,7,120,5873,0,task
105	103656142229,DEQ_WQ,7,120,5875,1,task
106	103656142312,ENQ_RQ,7,120,5875,1,task
107	103656142397,MUT_UL,7,120,5873,0,task
108	103656142554,SWT_AY,7,120,5873,0,task
109	103656142580,SWT_TO,7,120,5875,0,task
110	103656142972,MUT_LK,7,120,5875,0,task
111	104703866602,DEQ_WQ,7,120,5874,1,task
112	104703867211,ENQ_RQ,7,120,5874,1,task
113	104703867540,MUT_UL,7,120,5875,0,task
114	104703868768,SWT_AY,7,120,5875,0,task
115	104703868961,SWT_TO,7,120,5874,0,task
116	104703871281,MUT_LK,7,120,5874,0,task
117	105744740317,SWT_AY,7,120,5874,0,task
118	105744740949,SWT_TO,2,0,4129,0,rtdkit-daemon
119	105744957813,SWT_AY,2,0,4129,1,rtdkit-daemon
120	105744957869,SWT_TO,7,120,5874,0,task
121	105752209934,MUT_UL,7,120,5874,0,task
122	105752214814,DEQ_RQ,7,120,5874,1,task
123	105752214930,SWT_AY,7,120,5874,1,task
124	105752214981,SWT_TO,7,120,5875,0,task
125	105752226200,DEQ_RQ,7,120,5875,1,task
126	105752226323,SWT_AY,7,120,5875,1,task
127	105752226357,SWT_TO,7,120,5873,0,task

128	105752238349,DEQ_RQ,7,120,5873,1,task
129	105752240178,SWT_AY,7,120,5873,1,task
130	105752240212,SWT_TO,0,120,308,0,kworker/u2:5
131	112596217723,ENQ_RQ,7,120,5873,1,task
132	112596228955,SWT_AY,0,120,0,0,swapper
133	112596229051,SWT_TO,7,120,5873,0,task
134	112596260362,MUT_LK,7,120,5873,0,task
135	113601249053,ENQ_RQ,7,120,5874,1,task
136	113601249762,ENQ_RQ,7,120,5875,1,task
137	113601276445,SWT_AY,7,120,5873,0,task
138	113601276491,SWT_TO,7,120,5875,0,task
139	113601295970,ENQ_WQ,7,120,5875,1,task
140	113601296193,DEQ_RQ,7,120,5875,1,task
141	113601296316,SWT_AY,7,120,5875,1,task
142	113601296349,SWT_TO,7,120,5874,0,task
143	113601303493,ENQ_WQ,7,120,5874,1,task
144	113601303613,DEQ_RQ,7,120,5874,1,task
145	113601303727,SWT_AY,7,120,5874,1,task
146	113601303761,SWT_TO,7,120,5873,0,task
147	113634626953,DEQ_WQ,7,120,5874,1,task
148	113634627041,ENQ_RQ,7,120,5874,1,task
149	113634627121,MUT_UL,7,120,5873,0,task
150	113634627290,SWT_AY,7,120,5873,0,task
151	113634627316,SWT_TO,7,120,5874,0,task
152	113634627954,MUT_LK,7,120,5874,0,task
153	114684859412,DEQ_WQ,7,120,5875,1,task
154	114684860210,ENQ_RQ,7,120,5875,1,task
155	114684860459,MUT_UL,7,120,5874,0,task
156	114684861845,SWT_AY,7,120,5874,0,task
157	114684862047,SWT_TO,7,120,5875,0,task
158	114684864815,MUT_LK,7,120,5875,0,task
159	115738721701,MUT_UL,7,120,5875,0,task
160	115738797950,DEQ_RQ,7,120,5875,0,task
161	115738798818,ENQ_RQ,7,120,5875,0,task
162	115738807926,DEQ_RQ,7,120,5875,128,task
163	115738808673,SWT_AY,7,120,5875,128,task
164	115738808880,SWT_TO,7,120,5874,0,task
165	115738848423,DEQ_RQ,7,120,5874,0,task
166	115738848551,ENQ_RQ,7,120,5874,0,task
167	115738849934,DEQ_RQ,7,120,5874,128,task
168	115738850027,SWT_AY,7,120,5874,128,task
169	115738850054,SWT_TO,7,120,5873,0,task
170	115738883717,DEQ_RQ,7,120,5873,0,task
171	115738883839,ENQ_RQ,7,120,5873,0,task
172	115738884990,DEQ_RQ,7,120,5873,128,task
173	115738886921,SWT_AY,7,120,5873,128,task
174	115738886951,SWT_TO,0,120,4582,0,gvfs-afc-volume